О! Хакатон 2026 · трек «О!дин запрос»

GoGlobe

AI-агент для путешествий: архитектура, распределение ролей и роадмап разработки

One request. A trip that makes sense.

Команда: 5 человек — 2 Frontend · 1 дизайнер · 2 Backend
Хакатон: 23–30 октября 2026, онлайн · сдача решения до 29 октября, 10:00 (МСК)
Версия документа: 18 октября 2026

01 Хакатон и наш трек

Организатор — Островок · платформа Хакрус

8 дней
с 23 по 30 октября, онлайн

до 5
человек в команде — мы в полном составе

250 000 ₽
за 1 место в треке (150 000 ₽ и 100 000 ₽ — за 2 и 3)

1 млн ₽
общий призовой фонд, по 500 000 ₽ на трек

- **18 окт** — сегодня — подготовка
- **23 окт** — открытие, формат
- **24 окт** — работа над решением
- **25 окт** — обсуждение задач с экспертами
- **26 окт** — работа над решением
- **27 окт** — консультации с экспертами
- **28 окт** — финальная доработка
- **29 окт** — **сдача до 10:00 МСК**, финалисты
- **30 окт** — финал и награждение

Задача трека «О!дин запрос»

Создать **AI-агента для путешествий и отелей**, который берёт на себя рутинные задачи пользователя: понимает запрос и помогает подобрать подходящий вариант. Участникам трека доступны промокоды на бронирование отелей.

Тема хакатона — тревел-тех: переосмыслить планирование и бронирование путешествий с помощью AI. Целевая аудитория — Go- и Python-разработчики, аналитики, продакты.

Чего нет на странице — спросить у организаторов и экспертов

- Обязательный функционал, API и источники данных
- Критерии оценки и их веса
- Формат и длительность защиты, состав жюри
- Ограничения по технологиям и моделям
- Условия и размер промокодов

Источник: страница хакатона. Детали, вероятно, выдадут в регистрации (hackrus.ru/129401) или на открытии.

Идея продукта

ПРИМЕР ЗАПРОСА ПОЛЬЗОВАТЕЛЯ

«Хочу на 5 дней в Турцию в ноябре, на двоих, бюджет до 120 000 рублей. Нужен хороший отель рядом с морем и прямой перелёт»

Что агент извлекает из текста

направление: Турция | даты: ноябрь, 5 дней | путешественников: 2 | бюджет: до 120 000 ₽ | отель у моря | прямой перелёт

Если важного параметра не хватает, агент задаёт уточняющий вопрос вместо того, чтобы угадывать.

Что получает пользователь

Не просто список отелей, а **подборку под свой запрос с объяснением выбора**: карточки путешествий с ценой, отелем, перелётом, рейтингом и ссылкой на бронирование.

Принципы, на которых строим

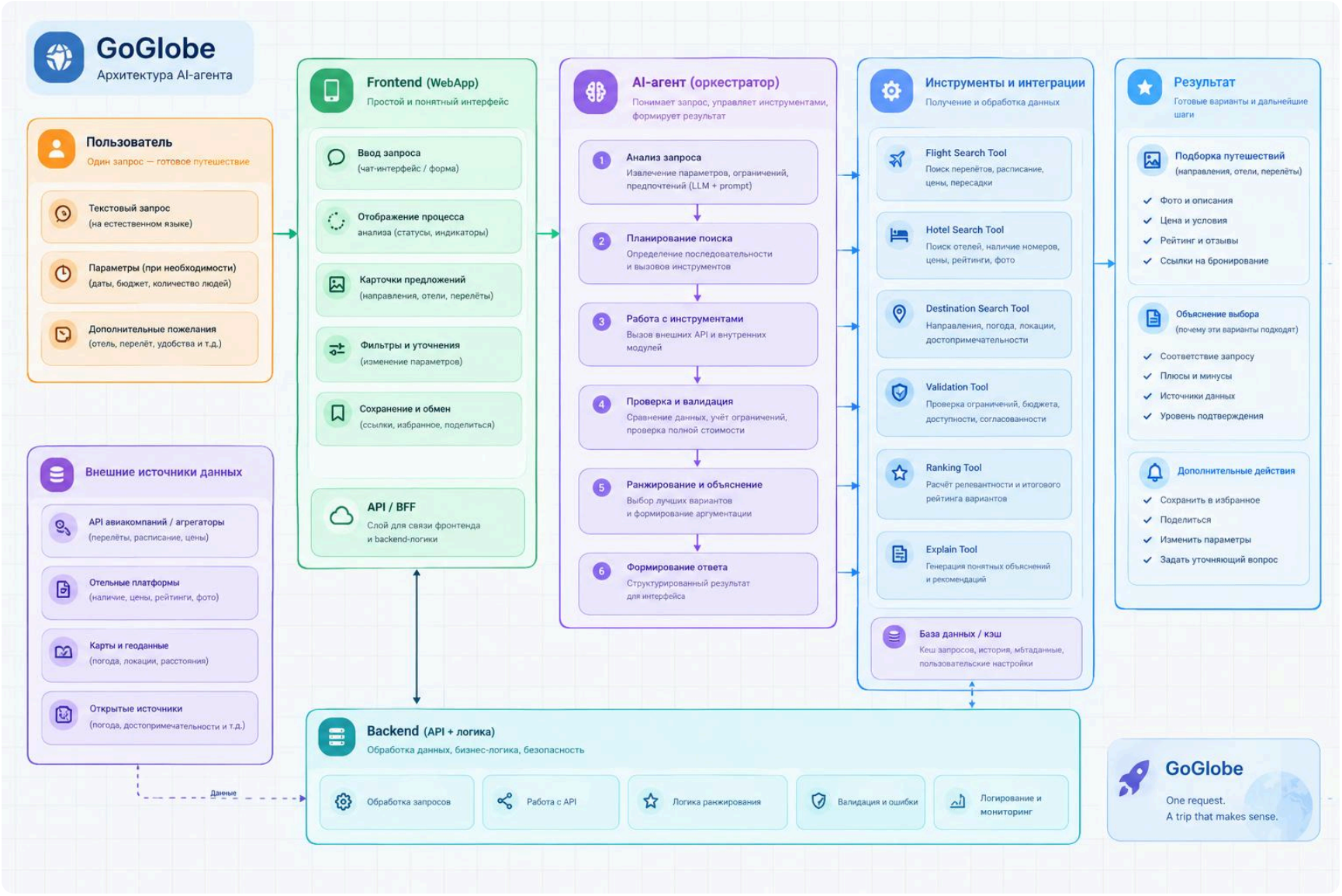
- **Честность данных.** AI не придумывает цены, наличие мест и характеристики отелей — только данные из источников. Если информации не хватает, система так и говорит.
- **Объяснимость.** У каждого варианта есть «почему подходит»: плюсы, минусы, источник, уровень подтверждения.
- **Один рабочий сценарий важнее десяти экранов.** К концу должен работать путь от запроса до подборки.
- **Запасной вариант для демо.** Любой внешний источник может подвести — на защите это не должно быть заметно.

Чем выигрываем на треке

- Один естественный запрос вместо десятка фильтров
- Реальные данные, а не заранее подготовленные карточки
- Проверка бюджета и дат до показа результата
- Прозрачное ранжирование: видно, почему вариант первый
- Аккуратный интерфейс с состояниями загрузки, ошибок и пустого результата

03 Архитектура AI-агента

Схема системы целиком



03 Из чего состоит система

Слои архитектуры и их задачи

Пользователь	Один запрос на естественном языке. По желанию — параметры (даты, бюджет, количество людей) и дополнительные пожелания (отель, перелёт, удобства).
Frontend (WebApp)	Ввод запроса (чат или форма), отображение процесса анализа (статусы, индикаторы), карточки предложений, фильтры и уточнения, сохранение и обмен (ссылки, избранное).
API / BFF	Слой связи фронтенда и backend-логики: единая точка входа, схемы запросов и ответов, передача статусов выполнения.
AI-агент (оркестратор)	Понимает запрос, управляет инструментами, формирует результат. Шесть шагов: анализ запроса → планирование поиска → работа с инструментами → проверка и валидация → ранжирование и объяснение → формирование структурированного ответа.
Инструменты и интеграции	**Flight Search** (перелёты, расписание, цены, пересадки) · **Hotel Search** (наличие номеров, цены, рейтинги, фото) · **Destination Search** (направления, погода, локации, достопримечательности) · **Validation** (ограничения, бюджет, доступность) · **Ranking** (релевантность и итоговый рейтинг) · **Explain** (понятные объяснения и рекомендации).
База данных / кэш	Кэш запросов, история, метаданные, пользовательские настройки. Экономит лимиты внешних API и страхует демо.
Backend (API + логика)	Обработка запросов, работа с API, логика ранжирования, валидация и ошибки, логирование и мониторинг.
Внешние источники данных	API авиакомпаний и агрегаторов, отельные платформы, карты и геоданные, открытые источники (погода, достопримечательности).
Результат	Подборка путешествий (фото, описание, цена, условия, рейтинг, ссылка на бронирование), объяснение выбора (соответствие запросу, плюсы и минусы, источники, уровень подтверждения) и дополнительные действия (избранное, поделиться, изменить параметры, задать уточняющий вопрос).

04 Как всё работает вместе

От запроса до готовой подборки — и кто за какой шаг отвечает

ШАГ 1
Пользователь вводит запрос
Например, про 5 дней в Турции в ноябре на двоих с бюджетом до 120 000 ₽.

Frontend №1
Дизайнер

ШАГ 2
Запрос уходит на сервер
Приложение передаёт запрос на backend, сервер принимает его и отдаёт AI-агенту.

Backend №1

ШАГ 3
AI понимает, что нужно
Выделяет направление, даты, бюджет, число людей и пожелания. Не хватает параметра — задаёт уточняющий вопрос.

Backend №2

ШАГ 4
Система ищет реальные варианты
Backend обращается к сервисам перелётов, отелей и направлений и получает цены, даты, характеристики и доступность.

Backend №1 — данные
Backend №2 — план поиска

ШАГ 5
Проверка и сравнение
Подходят ли варианты по бюджету, датам и пожеланиям; какие лучше всего. Затем ранжирование.

Backend №1 — корректность данных
Backend №2 — логика подбора

ШАГ 6
Пользователь видит подборку
Карточки путешествий, цены, отели, перелёты, объяснение выбора и ссылки на бронирование.

Frontend №2 — карточки
Frontend №1 — экраны

ШАГ 7
Пользователь выбирает вариант
Открывает детали, меняет параметры, сохраняет предложение или переходит по ссылке на бронирование.

Frontend — действия
Backend — запросы и данные

Не хватает параметра
Агент задаёт уточняющий вопрос — возврат к шагу 3, а не догадка.

Источник недоступен
Таймаут и повтор, затем кэш или запасные данные для демо с пометкой.

Ничего не найдено
Честное «пусто» и предложение изменить параметры — без выдуманных вариантов.

Путь данных в одну строку

Запрос → Frontend: интерфейс и ввод → API / BFF → AI-агент + внешние источники → Проверка и ранжирование → Карточки вариантов

Команда и распределение ролей

5 человек · зоны ответственности · точки интеграции



05 Роли: интерфейс и дизайн

Frontend №1
Основной интерфейс

Главная задача: сделать экраны, с которыми пользователь взаимодействует напрямую.

- Главный экран GoGlobe, навигация, роутинг
- Поле, куда пользователь вводит запрос обычным языком
- Форма доп. параметров: бюджет, даты, число путешественников, пожелания
- Кнопка запуска поиска
- Экран ожидания с понятным отображением процесса поиска
- Отображение ошибок
- Адаптация под телефон и компьютер

Результат: пользователь открывает приложение, вводит запрос и понимает, что происходит дальше.

Frontend №2
Результаты поиска

Главная задача: показать найденные варианты путешествий в удобном виде.

- Карточки найденных путешествий
- Отображение отелей, перелётов и направлений
- Страницы с подробной информацией о варианте
- Фильтры и сортировка результатов
- Избранное и возможность поделиться
- Цена, фотографии, рейтинги, ссылки на бронирование
- Подключение экранов к API от backend

Результат: пользователь получает понятный список, может сравнить варианты и выбрать подходящий.

Дизайнер
Внешний вид и удобство

Главная задача: продумать, как приложение выглядит и насколько удобно им пользоваться.

- Общий стиль GoGlobe: цвета, шрифты, иконки, карточки
- Расположение элементов на каждом экране
- Макеты главного экрана, поиска, результатов и деталей
- Путь пользователя: куда нажать и что произойдёт
- Как выглядят загрузка, ошибки и пустые результаты
- Передача макетов и графики frontend-разработчикам

Результат: оба frontend-разработчика делают интерфейс в едином стиле.

Важно: не просто красивые экраны, а путь пользователя от запроса до выбора.

05 Роли: серверная часть и AI

Backend №1

Сервер, данные и внешние сервисы

Главная задача: обеспечить получение настоящих данных о путешествиях и передачу их в приложение.

- Серверная часть, через которую приложение получает данные
- API — интерфейс общения frontend с сервером (API/BFF)
- Подключение внешних сервисов: авиабилеты, отели, направления
- Адаптеры: данные из сервисов приводятся к единому формату
- Обработка случаев, когда сервис не отвечает или возвращает ошибку; таймауты и повторы
- Хранение нужных данных и кэширование, чтобы не повторять одинаковые запросы
- Передача результатов AI-агенту и интерфейсу в согласованном формате

Результат: приложение получает реальные данные, а не показывает только заранее подготовленные карточки.

Важно: доступность API, условия использования и возможность получать цены и наличие нужно проверить заранее. Если источник недоступен — нужен запасной вариант для демонстрации.

Backend №2

AI-агент и логика подбора

Главная задача: сделать так, чтобы GoGlobe понимал запрос пользователя и подбирал подходящие варианты.

- Принимает запрос и преобразует в параметры: направление, даты, бюджет, число людей, пожелания
- Определяет, какие данные искать и какие инструменты использовать
- Организует поиск перелётов, отелей и других данных через backend и внешние сервисы
- Проверяет соответствие найденного запросу, отбрасывает неподходящее (слишком дорого, не те даты)
- Сравнивает оставшиеся варианты и ранжирует по полезности
- Объясняет, почему вариант подходит
- Формирует структурированный результат для карточек во frontend

Результат: пользователь получает не просто список отелей, а подборку под запрос с объяснением выбора.

Важно: AI не должен придумывать цены, наличие мест или характеристики отелей. Эти сведения берутся только из источников данных; если информации не хватает, система честно это показывает.

06 Как участникам работать друг с другом

Правила, чтобы не тратить время на переделки

Договорённости между ролями

- **Дизайнер → оба frontend.** Передаёт макеты, размеры, цвета, состояния экранов и компоненты.
- **Frontend №1 ↔ Backend №1.** Согласуют формат отправки запроса и формат ответа сервера.
- **Frontend №2 ↔ Backend №1.** Согласуют, какие поля нужны карточкам и страницам подробностей.
- **Backend №1 ↔ Backend №2.** Согласуют, как AI запрашивает данные, какие инструменты доступны и в каком формате возвращаются результаты.
- **Все пятеро.** Проверяют, что отдельные части соединяются в один рабочий сценарий.

Главный принцип

К концу работы должен существовать хотя бы **один полностью работающий сценарий**: пользователь вводит запрос, система получает данные, подбирает варианты и показывает результат. Это ценнее, чем десять красивых экранов, которые не связаны между собой.

Что делать в первую очередь

№	Кто	Что
1	**Вся команда**	Согласовать один демонстрационный сценарий и определить, что точно должно работать на защите
2	**Дизайнер**	Макеты только ключевых экранов, не проектируя сразу всё приложение
3	**Backend №1**	Проверить доступные источники данных и подготовить API-контракт: что принимает сервер и что возвращает
4	**Backend №2**	Обработка запроса, вызов инструментов, проверка и ранжирование вариантов
5	**Frontend №1**	Ввод запроса и отображение состояния поиска
6	**Frontend №2**	Карточки результатов и подробная страница варианта
7	**Вся команда**	Соединить части и проверить путь от одного запроса до готовой подборки

06 Матрица ответственности

Шаги сценария × роли: **О** — отвечает, **У** — участвует

Шаг сценария	Frontend №1	Frontend №2	Дизайнер	Backend №1	Backend №2
1. Пользователь вводит запрос	**О**		У		
2. Запрос отправляется на сервер	У			**О**	У
3. AI понимает, что нужно пользователю				У	**О**
4. Система ищет реальные варианты				**О (подключение и данные)**	**О (что и в каком порядке искать)**
5. Проверка и сравнение вариантов				**О (корректность данных)**	**О (соответствие запросу, подбор)**
6. Пользователь видит подборку	**О (экраны)**	**О (карточки, детали)**	У	У	У (формат результата)
7. Пользователь выбирает вариант	**О**	**О**		**О (запросы и сохранение)**	
Сквозное Дизайн-система и макеты	У	У	**О**		
Сквозное API-контракт	У	У		**О**	У
Сквозное Сборка демо и защита	У	У	У	У	У

Матрица собрана из распределения ролей и разбора сценария выше. Строки «Сквозное» добавлены как рабочее предложение: у каждой общей зоны должен быть один ответственный.

07 API-контракт: черновик v0

Предложение для согласования Frontend ↔ Backend №1 ↔ Backend №2

Запрос — POST /api/search

```
// один текст обязателен, параметры — по желанию
{
  "query": "5 дней в Турцию в ноябре, на двоих,
            до 120 000 ₽, отель у моря, прямой рейс",
  "params": {
    "dates":      { "from": "2026-11-10", "to": "2026-11-15" },
    "budget":     { "max": 120000, "currency": "RUB" },
    "travelers":  2,
    "wishes":     ["sea_view", "direct_flight"]
  }
}
```

Статусы поиска — GET /api/search/{id}/events (поток)

```
// Frontend №1 рисует «процесс анализа»
{ "stage": "parsing | flights | hotels | validating | ranking",
  "message": "Ищу отели у моря…" }
```

Ответ — итоговый результат

```
{
  "status": "ok | needs_clarification | partial | empty | error",
  "parsed": { /* что агент понял из запроса */ },
  "clarifying_question": "На какие даты в ноябре?",
  "offers": [{
    "id": "offer_1",
    "destination": "Анталья, Турция",
    "hotel":  { "name", "stars", "rating", "photos"[], "price" },
    "flight": { "direct": true, "dates", "price" },
    "total_price": { "amount": 112400, "currency": "RUB" },
    "explanation": {
      "why": ["в бюджете", "первая линия"],
      "pros": [], "cons": [],
      "sources": [], "confidence": "confirmed | estimated"
    },
    "booking_url": "https://…",
    "data_freshness": "2026-10-27T12:00Z"
  }],
  "warnings": ["Нет данных о наличии для 2 вариантов"]
}
```

Зачем такие поля. `status` и `warnings` дают дизайнеру состояния «пусто», «частично», «ошибка»; `confidence` и `sources` реализуют принцип «AI ничего не выдумывает»; `data_freshness` объясняет, откуда цены. Точный набор полей определяется тем, что реально отдают доступные API.

08 Роадмап: от подготовки до финала

18–30 октября 2026 · по ролям

	18–22 окт подготовка	23 окт старт	24–25 окт ядро и первый сквозной	26–27 окт реальные данные	28 окт полировка	29 окт	30 окт
Вся команда	Демо-сценарий, стек, репозиторий, договорённости	Открытие, формат, скелет проекта	Сквозной сценарий на моках (к вечеру 25-го)	Реальные данные, интеграция, заморозка функций (вечер 27-го)	Баги, питч, видео-запись демо	Сдача до 10:00	Финал
Дизайнер	Ключевые экраны, дизайн-система v0	Передача макетов frontend	Состояния: загрузка, ошибка, пусто; мобильные экраны	Ревью UI, ассеты, слайды питча	Питч-дек, сценарий демо		Питч
Backend №1	Проверка API и источников, API-контракт v0	Каркас API/BFF, мок-ответы	Адаптеры: перелёты, отели, направления; единый формат	Кэш, таймауты, повторы, запасные данные для демо	Стабильность, логи, деплой		
Backend №2	Схема параметров, промпты, список инструментов	Каркас агента	Разбор запроса и оркестрация на моках	Валидация бюджета, ранжирование, объяснения	Тесты на демо-запросах, защита от выдумок		
Frontend №1	Каркас WebApp, роутинг, форма	Ввод запроса	Статус поиска, ошибки, адаптив	Подключение реального API, уточняющие вопросы	Адаптив, полировка		
Frontend №2	Компоненты карточек на моках	Карточки результатов	Страница деталей, фильтры, сортировка	Избранное, шаринг, реальные данные	Полировка, пустые состояния		

Даты 25 и 27 октября — обсуждение задач и консультации с экспертами: приходим с готовым сквозным сценарием и списком вопросов. Деление по дням — рабочее предложение, расписание хакатона — с официальной страницы.

08 Контрольные точки и ритм работы

Как понимать, что мы на треке

22 ОКТЯБРЯ

M0 · Готовность к старту

Демо-сценарий утверждён. API-контракт v0 и макеты ключевых экранов готовы. Проверено, какие источники данных реально доступны. Репозиторий и окружение у всех.

25 ОКТЯБРЯ

M1 · Сквозной сценарий на моках

Запрос → статусы → карточки → детали работают целиком, пока на подготовленных данных. Есть что показать экспертам.

27 ОКТЯБРЯ, ВЕЧЕР

M2 · Реальные данные

Тот же сценарий на реальных источниках. Заморозка функций: дальше только исправления и полировка.

28 ОКТЯБРЯ, ВЕЧЕР

M3 · Демо готово

Стабильный деплой, запасные данные, запись демо-видео, питч-дек и репетиция выступления.

29 ОКТЯБРЯ, 10:00 МСК

M4 · Сдача

Решение отправлено с запасом по времени — лучше утром, а не в последние минуты.

Командные ритуалы

- Короткая синхронизация утром и вечером (15 минут): что сделано, что мешает, что нужно от других
- Ежедневный прогон сквозного сценария — каждый вечер, один и тот же запрос
- Один репозиторий, защищённая основная ветка, мелкие PR и быстрый ревью
- Контракт API меняется только через общий чат: изменил — сообщил всем

MVP и что сверх него

MVP (обязательно): ввод запроса, разбор параметров, поиск отелей и перелётов, проверка бюджета, ранжирование с объяснением, карточки и страница деталей, ссылка на бронирование.

Если останется время: уточняющие вопросы, фильтры и сортировка, избранное и шаринг, направления и погода, мобильная версия, история запросов.

09 Риски и запасные варианты

Что может пойти не так и что делаем заранее

Риск	Что делаем
Нет доступа к API или данные без цен и наличия	Backend №1 проверяет источники до 22 октября. Спрашиваем организаторов, какие данные и промокоды дают участникам трека. Запасной вариант: сохранённые реальные ответы (снимки) с честной пометкой даты данных.
AI выдумывает цены или характеристики	Цены и наличие берутся только из ответов инструментов. Агент возвращает строгую структуру, Validation Tool проверяет её. Нет данных — показываем «нет данных», а не догадку.
Медленный ответ модели или внешних API	Стриминг статусов вместо пустого экрана, таймауты и повторы, кэш запросов, заранее прогретые демо-запросы.
Части не склеиваются в конце	API-контракт v0 до старта, моки у обеих сторон, ежедневный сквозной прогон. Интеграция начинается 24 октября, а не 27-го.
Слишком большой объём, дизайнер и разработчики не успевают	Макеты только ключевых экранов. Заморозка функций вечером 27 октября. Всё из блока «если останется время» — только после M2.
Сбой на финальной демонстрации	Записанное демо-видео, заготовленные запросы, запасные данные, репетиция выступления 28 октября.
Не знаем критериев оценки и формата защиты	Уточняем у организаторов на открытии 23 октября и у экспертов 25 и 27 октября; питч строим вокруг рабочего сценария и честности данных.

10 Что сделать до старта: 18–22 октября

До открытия хакатона — 5 дней

Вся команда

- [ ] Утвердить один демо-сценарий и 2–3 запасных запроса
- [ ] Выбрать стек и создать общий репозиторий
- [ ] Договориться о каналах связи и времени синхронизаций
- [ ] Подготовить вопросы организаторам (критерии, данные, формат защиты, промокоды)
- [ ] Подтвердить регистрацию и состав команды на платформе

Дизайнер

- [ ] Дизайн-система v0: цвета, шрифты, иконки, карточка
- [ ] Макеты ключевых экранов: ввод, ожидание, результаты, детали
- [ ] Передать файлы и ассеты обоим frontend

Backend №1 и №2

- [ ] **№1:** проверить доступные API перелётов, отелей и направлений: условия, лимиты, наличие цен
- [ ] **№1:** черновик API-контракта и мок-ответы для фронтенда
- [ ] **№2:** схема параметров запроса, первые промпты, список инструментов агента
- [ ] **№2:** набор тестовых запросов с ожидаемым разбором
- [ ] Вместе: формат вызова инструментов и формат возврата результата

Frontend №1 и №2

- [ ] **№1:** каркас WebApp, роутинг, форма ввода на моках
- [ ] **№2:** компонент карточки и страница деталей на моках
- [ ] Вместе: общая структура проекта, подключение дизайн-токенов